

UNIDAD 9

ESTRUCTURAS DE DATOS AVANZADAS Y

ALGORITMOS

Programación
CFGs DAW

Autor: José Miguel Blázquez
2021/2022

Licencia



Reconocimiento - NoComercial - CompartirIgual (by-nc-sa):

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:

 Importante

 Atención

 Interesante

ÍNDICE DE CONTENIDO

1. Introducción.....	4
2. Estructuras de datos.....	4
2.1 Fundamentos de estructuras de datos.....	5
2.2 Estructuras de datos en Java.....	9
2.3 Bucle foreach.....	12
3. Listas: la clase LinkedList.....	12
4. Colas: la interfaz Queue.....	14
4.1 Colas de prioridad.....	15
5. Pilas: la clase Stack.....	15
5.1 Uso de la pila (stack).....	16
6. Mapas en Java: la clase HashMap.....	17
6.1 Creación de un HashMap.....	17
6.2 Métodos principales de HashMap.....	17
7. Conjuntos en Java: la interfaz Set.....	18
7.1 Uso de conjuntos en Java.....	19
7.2 Conjuntos ordenados.....	19
7.3 Aplicaciones a la teoría de conjuntos: unión, intersección y diferencia.....	20
8. Análisis y diseño con estructuras de datos.....	22
9. Algoritmos de ordenación.....	23
9.1 Ordenación por inserción directa.....	23
9.2 Ordenación por el método de la burbuja.....	24
9.3 Ordenación por QuickSort.....	25
9.4 Ordenación por MergeSort.....	26
9.5 Ordenación estándar en Java: el método .sort().....	27
10. Algoritmos de búsqueda.....	27
10.1 Búsqueda secuencial.....	27
10.2 Búsqueda binaria (dicotómica).....	28
11. Otros algoritmos.....	29
11.1 Técnicas y estrategias para diseño de algoritmos.....	29
11.2 Algoritmos del camino más corto y recubrimiento mínimo.....	31
11.2.1 Algoritmo de Dijkstra.....	31
11.2.2 Algoritmo de Kruskal.....	32
12. Complejidad computacional.....	33

UD9. ESTRUCTURAS DE DATOS AVANZADAS Y ALGORITMOS

1. INTRODUCCIÓN

Sin duda alguna, avanzar en el estudio del campo del desarrollo de software, y más concretamente de la programación, requiere estudiar y dominar los tipos de estructuras de datos más relevantes y los algoritmos que las manipulan.

Las estructuras de datos son el sistema mediante el cual se organizan los datos en la memoria de la aplicación que se está programando. Existen muchas formas de organizar los datos en la memoria, en esta unidad nos centraremos en las estructuras dinámicas y más avanzadas, al haber estudiado ya las básicas en unidades anteriores. Las estructuras de datos ofrecen una forma de almacenar y organizar la información para que se pueda usar de manera eficiente. Abarcan también el conjunto de algoritmos que podemos usar en cualquier lenguaje de programación para estructurar los datos en la memoria.

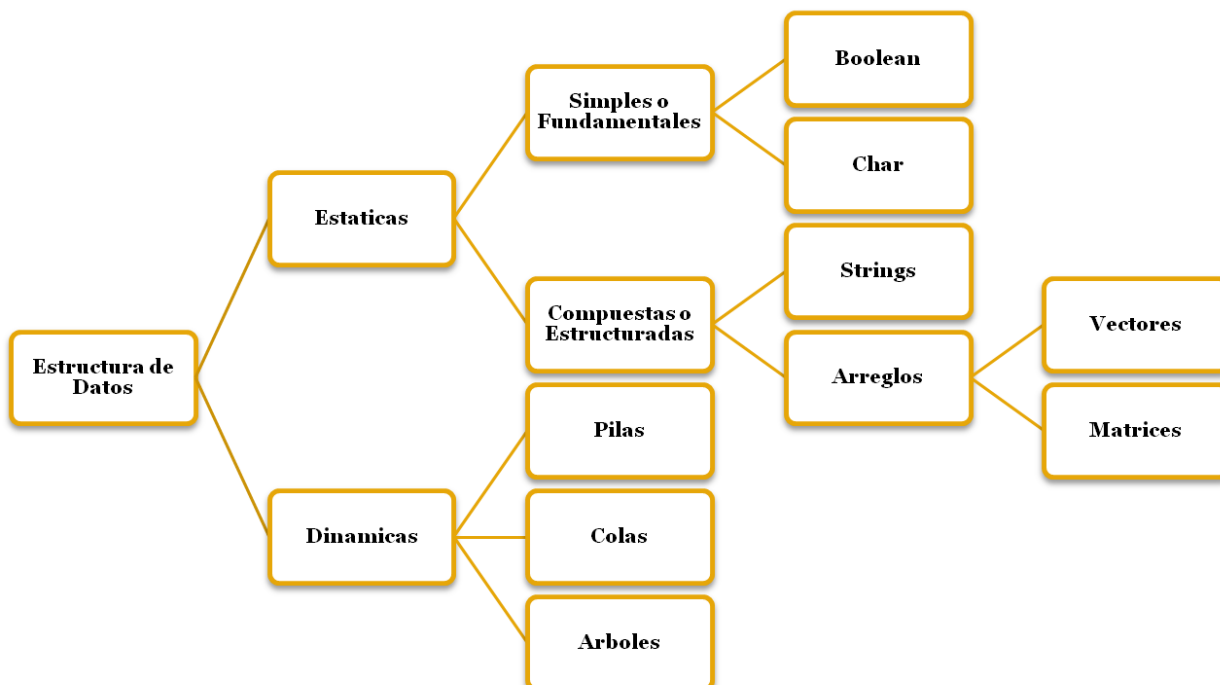
Los algoritmos son instrucciones precisas sobre cómo realizar una tarea específica. En el contexto de la programación los algoritmos se definen normalmente mediante el código fuente. La eficiencia de una sistema o aplicación determinada normalmente se mide o se asocia con los algoritmos. En un sentido más amplio, los algoritmos permiten recuperar y mostrar información rápidamente. Se trata de una parte fundamental dentro de las estructuras de datos.

2. ESTRUCTURAS DE DATOS

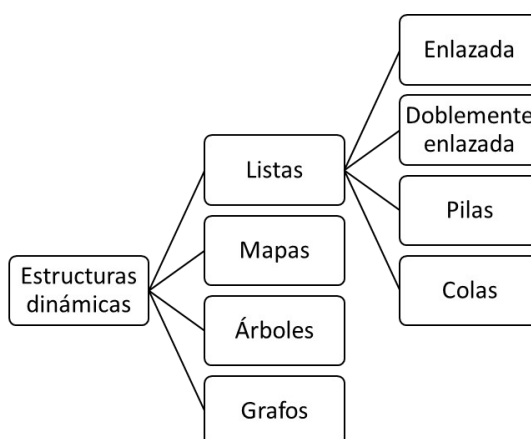
Una estructura de datos es una organización particular de una colección de elementos dados, con unas propiedades que la definen. Así, por ejemplo, una lista ordenada de números enteros almacenados en un array es un ejemplo de dicha estructura. La complejidad de la información que maneja una aplicación y los algoritmos que se emplean, hoy en día requieren, más que nunca, la existencia de esas estructuras.

Será necesario que la elección de una estructura de datos adecuada requiera también la posibilidad de poder realizar operaciones adecuadas sobre dichas estructuras. La **elección** de la estructura de datos adecuada al problema a resolver redundará en una mayor eficiencia del programa y, sobre todo, en una mejor resolución del problema en cuestión. Una elección inadecuada de la estructura de datos puede conducir a programas lentos, largos y poco eficientes.

Para situarnos en contexto vamos a recordar una clasificación de las estructuras de datos según si son datos primitivos o no. Algunas de estas ya las hemos trabajado en temas anteriores, por lo que en esta unidad nos centraremos en las novedades.



A continuación, una clasificación más detallada de las estructuras dinámicas a estudiar (desde un punto de vista teórico) en esta unidad.



2.1 Fundamentos de estructuras de datos

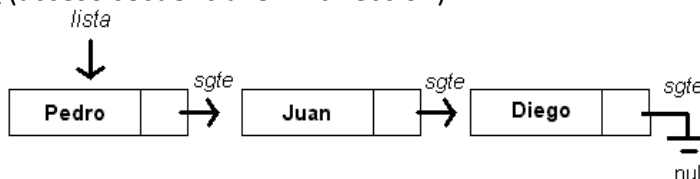
En este apartado, vamos a realizar una exposición teórica de las estructuras de datos desde un punto de vista matemático y abstracto, la cual nos servirá para entenderlas un poco mejor antes de pasar a utilizarlas en nuestras aplicaciones.

1. Listas

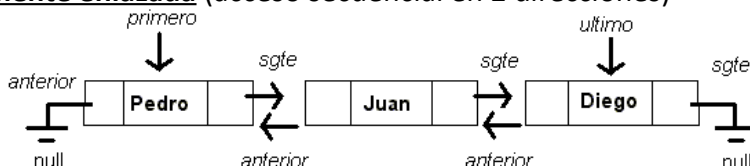
Las listas están formadas por un conjunto de elementos, normalmente homogéneos, que se recorren de manera secuencial. Cada elemento de la lista tiene asignado un índice según la posición que ocupa, de forma que, por lo general, para acceder a un elemento en la posición N, es necesario recorrer los N-1 elementos anteriores a él. Aspectos como por ejemplo si la lista está ordenada o no, pueden cambiar la forma de trabajar con ellas.

Subtipos de listas

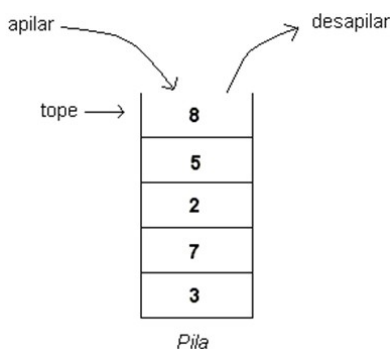
- **Enlazada** (acceso secuencial en 1 dirección)



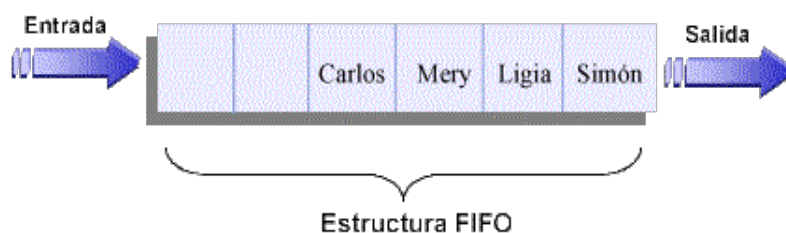
- **Doblemente enlazada** (acceso secuencial en 2 direcciones)



- **Pila**: particularidad de la lista enlazada donde los elementos se insertan y extraen desde el mismo extremo. Suelen implementar LIFO (Last In First Out). Algunas de sus aplicaciones son: almacenamiento en memoria de llamadas recursivas a funciones, la función “deshacer / rehacer”, mantenimiento de un listado de “archivos recientes”, la realización de operaciones en las calculadoras, etcétera...



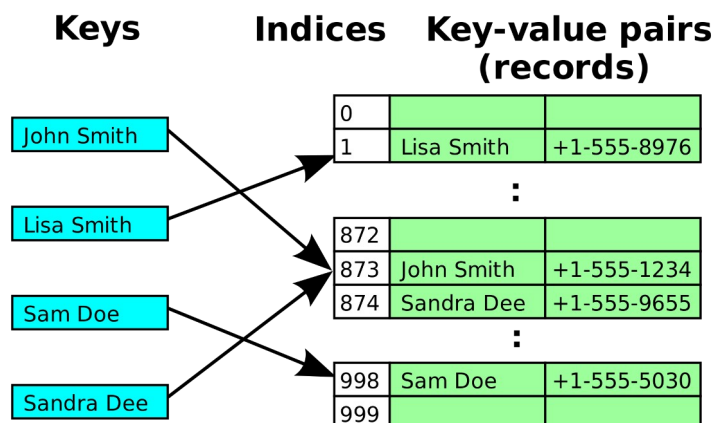
- **Cola**: tipo de lista con la particularidad de que sólo permite insertar elementos por un lado y extraerlos por el otro. Generalmente implementan FIFO (First In First Out). Algunas de sus aplicaciones son: gestionar prioridades (las listas de procesos a ejecutar por la CPU, la cola de la caja del supermercado, la cola de impresión...),



2. Mapas o tablas “hash”

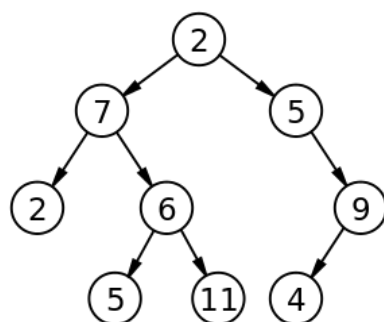
En estas estructuras los elementos se referencian por una clave en vez de por un índice numérico (como en los arrays). De esa forma, lo que se almacena en una tabla hash son pares de claves y valores (o registros) asociados, por lo que conociendo la clave podemos acceder directamente al valor asociado sin necesidad de recorrer la estructura. Esto se hace efectivo mediante una “función hash” que calcula la correspondencia de la clave al valor.

Por ejemplo, una tabla hash donde la clave sea el DNI y el valor asociado una estructura llamada “Persona” con los datos personales de una persona. O bien una donde, sabiendo el nombre, podamos acceder al registro del teléfono de una persona...para entender mejor su objetivo, piensa, ¿cómo almacenarías esto en un array normal?



3. Árboles

Esta estructura, ampliamente usada y que imita la forma de un árbol, representa un conjunto de nodos donde cada uno está enlazado con otros nodos llamados hijos formando una jerarquía no cíclica. Si cada nodo sólo puede tener como máximo 2 hijos (izquierdo y derecho) se llama árbol binario. Veamos un ejemplo gráfico del árbol binario:



Terminología básica:

- **Raíz:** El nodo superior e inicial de un árbol.
- **Hijo:** Un nodo conectado directamente con otro cuando se aleja de la raíz.
- **Padre:** La noción inversa de hijo.
- **Hermanos:** Un conjunto de nodos con el mismo padre.
- **Hoja:** Un nodo sin hijos.
- **Camino:** Una secuencia de nodos y brazos conectados con un nodo descendiente.

Si bien poseen las mismas operaciones que las listas (insertar, eliminar y buscar), la forma de recorrerlos puede ser diferente, siendo la manera más efectiva de hacer estos recorridos la recursiva. Veamos el orden de recorrido de los nodos de las formas más importantes:

- **Preorden:** raíz, izquierdo, derecho.
- **Inorden:** izquierdo, raíz, derecho.
- **Postorden:** izquierdo, derecho, raíz.

Los árboles se emplean en multitud de problemas, como por ejemplo, en los algoritmos de encaminamiento (ruta óptima en GPS).

Hay un caso particular de árbol que se llama “**heap**” o **montículo**. Es una estructura de datos muy relevante aunque sólo la vamos a mencionar brevemente. Se caracteriza por ser un árbol binario donde cada nodo tiene una ordenación siempre igual (o bien cada nodo padre tiene un valor mayor que el de cualquiera de sus nodos hijos o al revés, el valor del nodo padre es siempre menor al de sus nodos hijos). Éstos tienen multitud de aplicaciones, como por ejemplo, la implementación de colas de prioridad.

4. Grafos

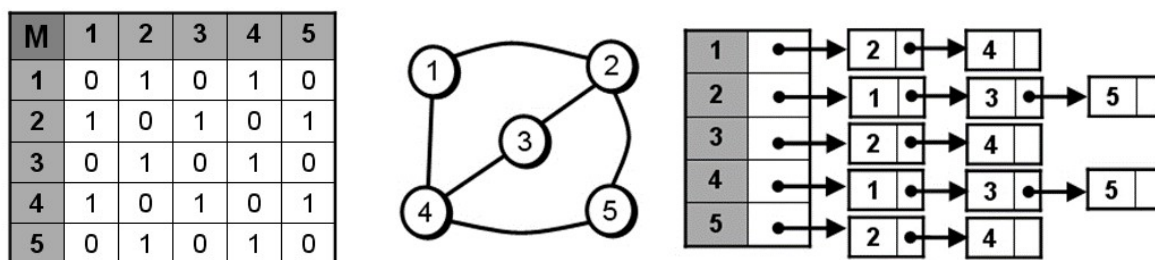
Estructuras matemáticas similares a los árboles formadas por un conjunto de nodos (vértices) y aristas (uniones entre 2 vértices) que pueden tener una distancia, un peso o un coste, y además, pueden ser dirigidos o no. Si sus aristas tienen peso se les llama

ponderados.

Las operaciones básicas a realizar sobre grafos son añadir y eliminar nodos y arcos. También podemos recorrer un grafo, de forma similar a un árbol, aunque mediante el sistema de arcos. En este caso, es habitual calcular el coste (suma de pesos) de un recorrido por el grafo para ir de un nodo a otro.

Infinidad de algoritmos emplean los grafos como estructura de datos. Podemos citar como ejemplos el algoritmo de Kruskal, el de Dijkstra, el de Ford-Fulkerson o el de Bellman-Ford. Los algoritmos de enrutamiento dinámico, empleados por los routers en el nivel IP de las redes de comunicaciones, emplean grafos para su funcionamiento y algoritmos como algunos de los mencionados.

Se les puede representar mediante matrices de adyacencia o listas de adyacencia. Veamos un ejemplo de grafo no dirigido representado en ambas formas, con matriz y con lista:



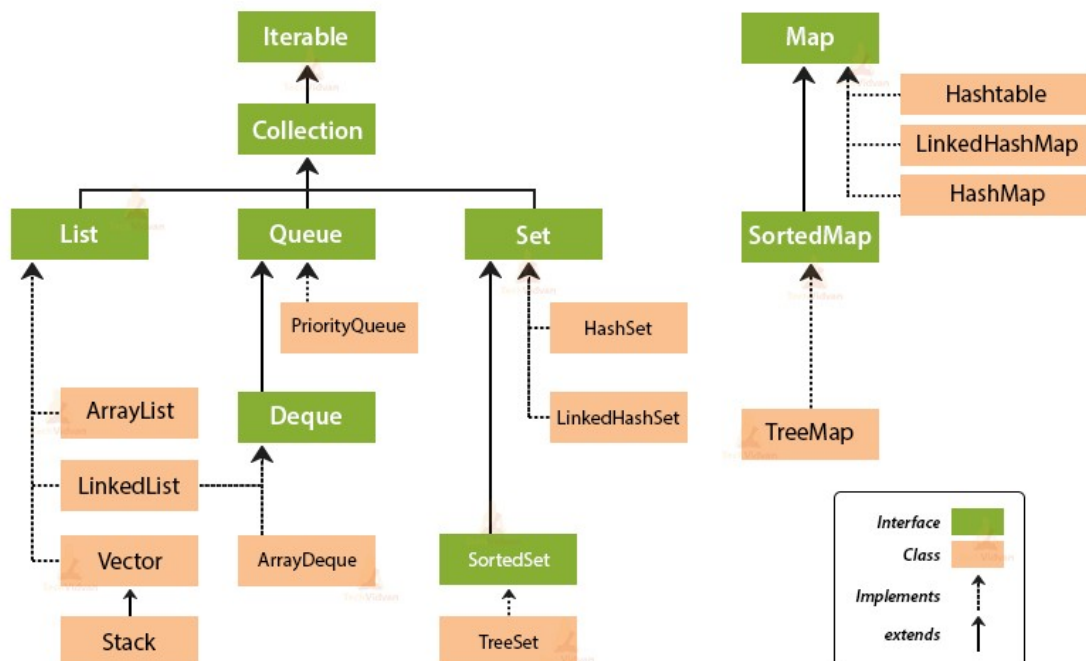
2.2 Estructuras de datos en Java

Una vez hemos conocido las estructuras de datos más relevantes, vamos a ver cómo implementar cada una mediante el lenguaje Java.

Es importante empezar resaltando que el framework de Colecciones en Java no proporciona clases estándar para modelar árboles o grafos, se deja a la responsabilidad del programador su implementación concreta mediante otras estructuras. Dentro de las colecciones existen 3 interfaces principales: **List**, **Set** y **Map**. Cuando se crea una clase que implementa alguna de estas interfaces, está obligada a tener los métodos que indica dicha interfaz.

Veamos en la siguiente imagen la jerarquía del framework "Collection":

Collection Framework Hierarchy in Java



De todas ellas, las estructuras más relevantes son:

- **ArrayList** (lista de acceso directo por índice tipo array): clase
- **LinkedList** (lista enlazada y doblemente enlazada): clase
- **Queue** (cola usando LinkedList): interfaz
- **Stack** (pila): clase
- **HashMap** (hereda de Map): clase
- **Set** (conjunto): interfaz

Veamos sus detalles:

Listas (List)

Las colecciones de tipo **List** son las más parecidas a los arrays y permiten instanciar objetos de clases hijas, como **ArrayList** y **LinkedList**. Dependiendo de la finalidad de la colección que necesitemos tenemos otros tipos de estructuras dinámicas: los conjuntos **Set**, y los mapas **Map**.

Conjuntos (Set)

Los conjuntos en Java, al igual que los conjuntos en matemáticas, no permiten contener

elementos duplicados. Implementan la interface Set:

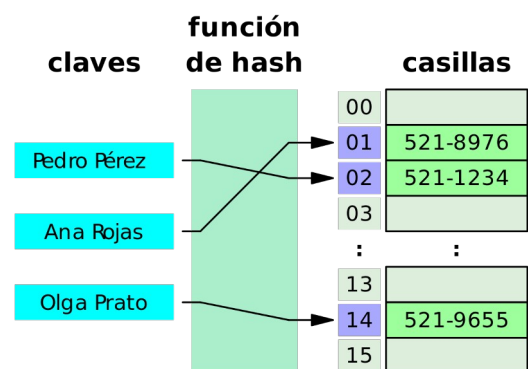
- **HashSet** : Almacena los elementos en una tabla hash sin mantener ningún orden.
- **TreeSet** : Almacena los elementos ordenándolos en función de sus valores. Necesita que los elementos implementen la interface `Comparable`
- **LinkedHashSet** : Almacena los elementos en función de su orden de inserción.

Mapas (Map)

Corresponden al concepto de diccionario o pares clave-valor, habitualmente con tabla hash. Son colecciones que contienen asociaciones *clave:valor*, esto quiere decir que al añadir un elemento a la colección debemos añadir una clave para identificarlo. Por ejemplo: `key` → `dni`, `value` → una persona. Estas estructuras implementan la interface `Map`

Para recuperar los elementos añadidos, debemos indicar la clave del elemento.

Key	Value
714435667R	Persona (nombre = "Juan", apellidos = "López")
985543264X	Persona (nombre = "Maria", apellidos = "Martinez")
98773255L	Persona (nombre = "Javier", apellidos = "López")



En Java, las clases más relevantes que implementan este concepto son:

- **HashMap**: almacena las claves en una tabla hash sin orden (la más habitual).
- **TreeMap** : almacena las claves ordenándolas en función de sus valores.
- **LinkedHashMap** : almacena las claves en función de su orden de inserción.

2.3 Bucle foreach

Conocido como `foreach` o `for` mejorado, es una versión particular del conocido bucle `for` enfocada en recorrer listas. **No se puede usar para borrar elementos.**

Se le indican 3 cosas:

- Tipo de los datos de la lista
- Variable con la que accedo a cada dato de la lista (la tradicional "i" en un `for` normal).
- Objeto lista a recorrer (Cualquier tipo de `Collection` o `Array`)

```
ArrayList<String> listaNombres = new ArrayList<String>();

listaNombres.add("Jose");
listaNombres.add("Marta");
listaNombres.add("Juan");
listaNombres.add("Laura");

for(String nombre: listaNombres){
    System.out.println(nombre);
}
```

3. LISTAS: LA CLASE LINKEDLIST

Implementa el concepto general de una lista, tanto la enlazada como la doblemente enlazada. La usamos cuando lo que queremos es insertar o borrar en una posición determinada y nos prima la eficiencia. Es una especie de `ArrayList` que no tiene definido una posición de inicio ni de fin, emplea básicamente lo mismos métodos y de la misma forma que `ArrayList` ya que son clases que implementan la interface **List** (orden superior a ambas).

```
LinkedList<String> lista1 = new LinkedList<String>();
List<String> lista1 = new LinkedList<String>();
```

Mediante el método '**add**' añadimos al final nodos:

```
lista1.add("juan");
lista1.add("Luis");
lista1.add("Carlos");
```

Llamamos al método estático '`imprimir`' para mostrar todos los elementos de la lista:

```
imprimir(lista1);
```

El método estático recibe la lista y mediante un `foreach` recorre la colección mostrando sus elementos:

```
public static void imprimir(LinkedList<String> lista) {  
    for (String elemento : lista)  
        System.out.print(elemento + "-");  
    System.out.println();  
}
```

El método 'add' de la clase LinkedList se encuentra sobrecargado existiendo un segundo método 'add' con 2 parámetros que recibe: en el primer parámetro la posición donde se debe insertar el nodo y como segundo parámetro el dato a almacenar:

```
lista1.add(1, "ana");
```

Para eliminar un nodo de la lista debemos llamar al método 'remove' y pasar la posición del nodo a eliminar:

```
lista1.remove(0);
```

También podemos eliminar los nodos que coinciden con cierta información:

```
lista1.remove("Carlos");
```

La cantidad de nodos nos lo suministra el método 'size':

```
System.out.println("Cantidad elementos en lista:" + lista1.size());
```

Para conocer si la lista almacena cierto valor disponemos del método 'contains':

```
if (lista1.contains("Ana"))  
    System.out.println("El nombre 'Ana' si esta en la lista");  
else  
    System.out.println("El nombre 'Ana' no esta en la lista");
```

Para recuperar el dato de un nodo sin eliminarlo podemos hacer uso del método 'get':

```
System.out.println("El 2º elemento de la lista es:" + lista1.get(1));
```

Eliminamos todos los nodos de la lista mediante el método 'clear':

```
lista1.clear();
```

Podemos consultar si la lista se encuentra vacía mediante el método 'isEmpty':

```
if (lista1.isEmpty())  
    System.out.println("La lista se encuentra vacía");
```

La clase LinkedList también nos permite tratar la lista como si fuera doblemente enlazada

gracias a determinados métodos de la interfaz `ListIterator`, como son: `.next()` y `.previous()`

¿Cuándo usar `LinkedList` y cuando `ArrayList`? Si se requiere agregar y borrar datos con mucha frecuencia, `LinkedList` será la mejor opción. En cambio, si hay pocas operaciones de agregación y borrado, y muchas de búsqueda y consulta, entonces mejor usar `ArrayList`.

4. COLAS: LA INTERFAZ `QUEUE`

Una cola FIFO, en Java, se puede implementar usando la interfaz “`Queue`” y la clase “`LinkedList`”, así podremos administrar una lista con la funcionalidad de una cola (dado que, como vimos al principio de la unidad, una cola es un tipo particular de lista).

```
Queue<String> cola1 = new LinkedList<String>();
```

La clase `LinkedList` implementa la interfaz `Queue` que es la que declara los métodos principales para trabajar una cola.

Añadimos elementos al final de la cola mediante el método '**add**':

```
cola1.add("juan");
cola1.add("ana");
cola1.add("luis");
```

Para conocer la cantidad disponemos del método **size()**:

```
System.out.println("Cantidad elementos en la cola:" + cola1.size());
```

Para extraer el nodo de principio de la lista tipo cola lo hacemos mediante el método '**poll**':

```
System.out.println("Extraemos elemento de la cola:" + cola1.poll());
```

Si queremos conocer el objeto primero de la cola sin extraerlo lo hacemos mediante el método '**peek**':

```
System.out.println("Consulta 1er elemento:" + cola1.peek());
```

Para conocer si la cola se encuentra vacía llamamos al método '**isEmpty**':

```
while (!cola1.isEmpty())
    System.out.print(cola1.poll() + "-");
```

Podemos utilizar un `foreach` para recorrer todos los elementos de la colección de tipo ***Queue*** con la siguiente sintaxis (no se eliminan los elementos de la cola):

```
for (Integer elemento : cola2)
    System.out.print(elemento + "-");
```

Para eliminar todos los elementos de una pila empleamos el método `clear`:

```
cola2.clear();
```

4.1 Colas de prioridad

En Java también existe la clase “`PriorityQueue`”, que permite instanciar directamente una cola de prioridad. **Ojo:** las colas de prioridad ya no respetan el FIFO estrictamente. Veamos un ejemplo:

```
PriorityQueue<Integer> cola1 = new PriorityQueue<Integer>();

cola1.add(70);
cola1.add(120);
cola1.add(6);

System.out.println("Imprimimos cola prioridades de enteros");
while (!cola1.isEmpty())
    System.out.print(cola1.poll() + "-");
```

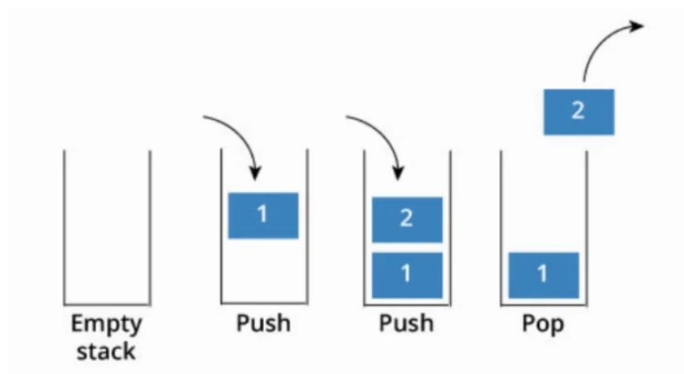
Cuando comenzamos a recuperar los elementos de la cola con prioridad podemos comprobar que el primero de la cola es el que tiene el valor 6, luego el 70 y finalmente el 120.

5. PILAS: LA CLASE STACK

Una pila (stack) es una estructura que funciona como una lista LIFO (Last In First Out). El último elemento añadido será el primero en salir. Se puede entender como una pila de platos, se van colocando uno encima de otro y se van cogiendo desde arriba hacia abajo, o de libros.

Tiene 5 métodos principales, además de todos los que posee la interface “`List`” ya que la implementa.

- **`push(Elemento)`** : **apilar**, añade un elemento a la pila
- **`pop()`** : **desapilar**, devuelve el último elemento de la pila y lo elimina de la pila
- **`empty()`** : indica si la pila está vacía
- **`peek()`** : devuelve el último elemento pero sin desapilarlo (lo consulta)
- **`search(Object)`** : indica la posición (int) del elemento “`Object`” en la pila



5.1 Uso de la pila (stack)

Debemos empezar instanciando (creando) una pila (2 ejemplos):

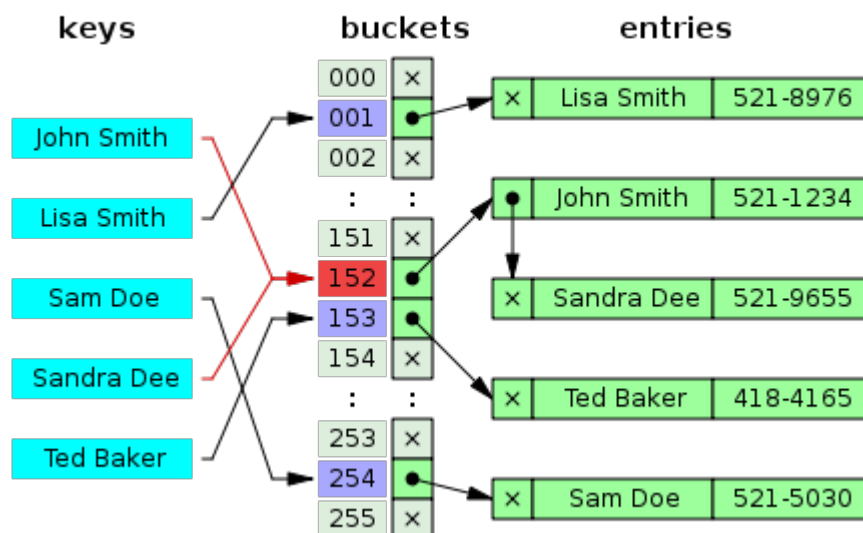
```
Stack<String> pilaStrings = new Stack<String>();  
Stack<Persona> pilaPersonas = new Stack<Persona>();
```

Ejemplo de uso de una pila:

```
import java.util.Stack;  
Stack<String> pila1 = new Stack<String>();  
  
//apilar 3 elementos (de tipo String)  
pila1.push("elemento1");  
pila1.push("elemento2");  
pila1.push("elemento3");  
  
//mostrar por consola  
for(String cadena : pila1){  
    System.out.println(cadena);  
}  
  
//elimina y devuelve el elemento que está en la cima  
String desapilado = pila1.pop();  
System.out.println(desapilado);  
  
//devuelve el elemento que está en la cima sin borrarlo  
String ultimo = pila1.peek();  
System.out.println(ultimo);
```


6. MAPAS EN JAVA: LA CLASE HASHMAP

De todos los mapas disponibles en Java la clase “HashMap” es una de las más utilizadas. En la teoría se les conoce también como tablas hash. Ejemplos de uso:



6.1 Creación de un HashMap

Para crear un objeto de la clase HashMap podemos hacerlo con la siguiente sintaxis:

```
Map<String, Persona> mapa1 = new HashMap<String, Persona>();
```

La clase HashMap utiliza datos genéricos tanto para la clave como para el valor, en este ejemplo la clave es String y el valor almacenado es un objeto de tipo Persona. También se puede instanciar así:

```
HashMap<String, Persona> mapa1 = new HashMap<String, Persona>();
```

Requiere: `import java.util.HashMap;`

6.2 Métodos principales de HashMap

Mediante el método **put** agregamos un elemento a la colección de tipo HashMap. Desconocemos donde lo va a ubicar físicamente ya que realiza un cálculo (función de hashing) para obtener el lugar donde situarlo:

```
mapa1.put("rojo", "red");
mapa1.put("verde", "green");
mapa1.put("azul", "blue");
mapa1.put("blanco", "white");
```

Para imprimir todos los **valores** del mapa lo recorremos mediante un **for**:

```
for (String valor : mapa1.values())  
    System.out.print(valor + "-");
```

De forma similar si queremos recorrer todas las **claves** del mapa:

```
for (String clave : mapa1.keySet())  
    System.out.print(clave + "-");
```

Para recuperar un valor para una determinada clave llamamos al método '**get**' y le pasamos la clave a buscar, si dicha clave no existe en el mapa nos devuelve el valor 'null':

```
System.out.println("A 'rojo' corresponde:" + mapa1.get("rojo"));
```

Si queremos verificar si una determinada clave existe en el mapa lo hacemos mediante el método '**containsKey**':

```
if (mapa1.containsKey("negro"))  
    System.out.println("No existe la clave 'negro'");
```

Una variante del método '**get**' es '**getOrDefault**' que nos retorna el segundo parámetro si no encuentra la clave en el mapa:

```
System.out.println("La traducción de 'marron' es:" +  
    mapa1.getOrDefault("marrón", "No existe la clave  
marrón"));
```

Para eliminar un elemento de la colección debemos hacer uso del método '**remove**' pasándole una clave del mapa:

```
mapa1.remove("rojo");
```

Para imprimir el mapa completo en la Consola podemos hacer uso del método '**println**':

```
System.out.println(mapa1);
```

7. CONJUNTOS EN JAVA: LA INTERFAZ SET

Los conjuntos son un tipo de estructura de datos que, si bien estructuralmente no se diferencian de una lista, funcionalmente sí ya que NO permiten elementos duplicados. En Java existe una interfaz que permite implementar conjuntos y, por ese motivo, y por el hecho de que son una estructura relevante, vamos a mencionarla. Un objeto **Set** es un objeto **Collection** que

contiene elementos únicos (no duplicados). El marco de trabajo de colecciones contiene diferentes implementaciones de Set incluyendo **HashSet** y **TreeSet**. El primero almacena los elementos en una tabla **hash** y el segundo en un **árbol**.

7.1 Uso de conjuntos en Java

En el siguiente ejemplo, primero creamos un HashSet y agregamos 2 elementos. Luego creamos otro HashSet y agregamos 2 elementos más. Esto luego se agrega como una colección al primer conjunto. En el ejemplo de eliminación, primero eliminamos un elemento del primer conjunto y luego eliminamos todos los elementos del segundo conjunto.

```
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.Set;
import java.util.TreeSet;

public class SetExample {

    public static void main(String[] args) {

        Set<String> setstring = new HashSet<String>();
        setstring.add("Java");
        setstring.add("Javascript");
        System.out.println(setstring);

        Set<String> string2 = new HashSet<String>();
        string2.add("C");
        string2.add("C++");

        setstring.addAll(string2);
        System.out.println("Elementos tras addAll");
        System.out.println(setstring);

        setstring.remove("Javascript");
        System.out.println("Elementos tras borrado");
        System.out.println(setstring);

        setstring.removeAll(string2);
        System.out.println("Elementos tras borrar todos");
        System.out.println(setstring);
    }
}
```

7.2 Conjuntos ordenados

El marco de trabajo de colecciones incluye también la interfaz **SortedSet** (que extiende a **Set**) para conjuntos que mantengan sus elementos ordenados, ya sea en el orden natural de los elementos o por un orden especificado por un objeto **Comparator**. La clase **TreeSet** implementa a **SortedSet**.

Veamos un ejemplo de implementación. Hemos declarado una matriz de enteros y luego agregamos los elementos al HashSet usando el bucle for. En la salida se puede ver que el

elemento duplicado "12" no está incluido, que es la característica establecida. Podemos ordenar los elementos de HashSet usando TreeSet:

```
import java.util.*;

public class SetSort {

    public static void main(String[] args) {

        Set<Integer> numbers = new HashSet<Integer>();
        int[] a = {34,78,12,67,45,23,12,89};
        for(int i=0;i<a.length;i++) {
            numbers.add(a[i]);
        }
        System.out.println("Elements in HashSet");
        System.out.println(numbers);

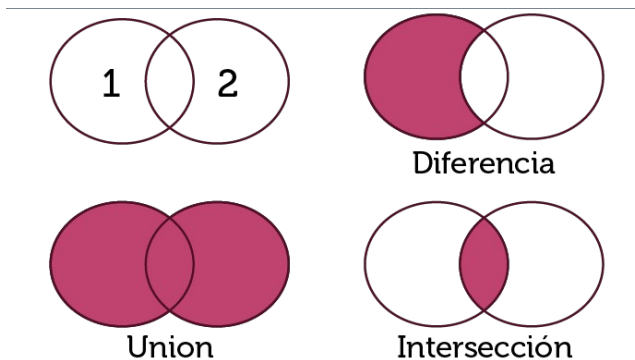
        Set<Integer> treeset = new TreeSet<Integer>(numbers);
        System.out.println("Elements in treeset after sort");
        System.out.println(treeset);
    }
}
```

7.3 Aplicaciones a la teoría de conjuntos: unión, intersección y diferencia

Los conjuntos tienen infinidad de aplicaciones, en principio todas aquellas donde en la propia matemática se emplean. Sin embargo, un gran campo de aplicación es la teoría de conjuntos; ampliamente utilizada en probabilidad y otros campos.

Para entender lo que vamos a realizar, y antes de la implementación, haremos una pequeña introducción. La teoría de conjuntos es una rama de la lógica matemática que estudia las propiedades y relaciones de los conjuntos: **colecciones** abstractas de objetos, consideradas como objetos en sí mismas. La teoría de los conjuntos es lo suficientemente rica como para construir el resto de objetos y estructuras de interés en matemáticas: números, funciones, figuras geométricas, etc.

Existen unas operaciones básicas que permiten manipular los conjuntos y sus elementos, similares a las operaciones aritméticas, constituyendo el álgebra de conjuntos, estas son: la unión, la intersección y la diferencia. Existen otras pero nos centraremos en estas últimas. Veamos un ejemplo gráfico que las ilustra, se llaman diagramas de Venn:



Veamos a continuación, en Java, como se puede emplear la interfaz Set y la clase HashSet para implementar la lógica de conjuntos anterior:

```
import java.util.*;

public class SetOperations {

    public static void main(String[] args) {

        Set<Integer> set1 = new HashSet<Integer>();
        set1.add(3);
        set1.add(7);
        set1.add(5);
        set1.add(1);
        System.out.println("Set 1 : " + set1);

        Set<Integer> set2 = new HashSet<Integer>();
        set2.add(3);
        set2.add(4);
        set2.add(9);
        set2.add(5);

        System.out.println("Set 2 : " + set2);

        Set<Integer> a = new HashSet<Integer>(set1);
        a.addAll(set2);

        System.out.println("Union de 1 y 2 : " + a);

        Set<Integer> b = new HashSet<Integer>(set1);
        b.retainAll(set2);
        System.out.println("Intersección de 1 y 2 : " + b);

        Set<Integer> c = new HashSet<Integer>(set1);
        c.removeAll(set2);
        System.out.println("Diferencia entre 1 y 2 : " + c);
    }
}
```

8. ANÁLISIS Y DISEÑO CON ESTRUCTURAS DE DATOS

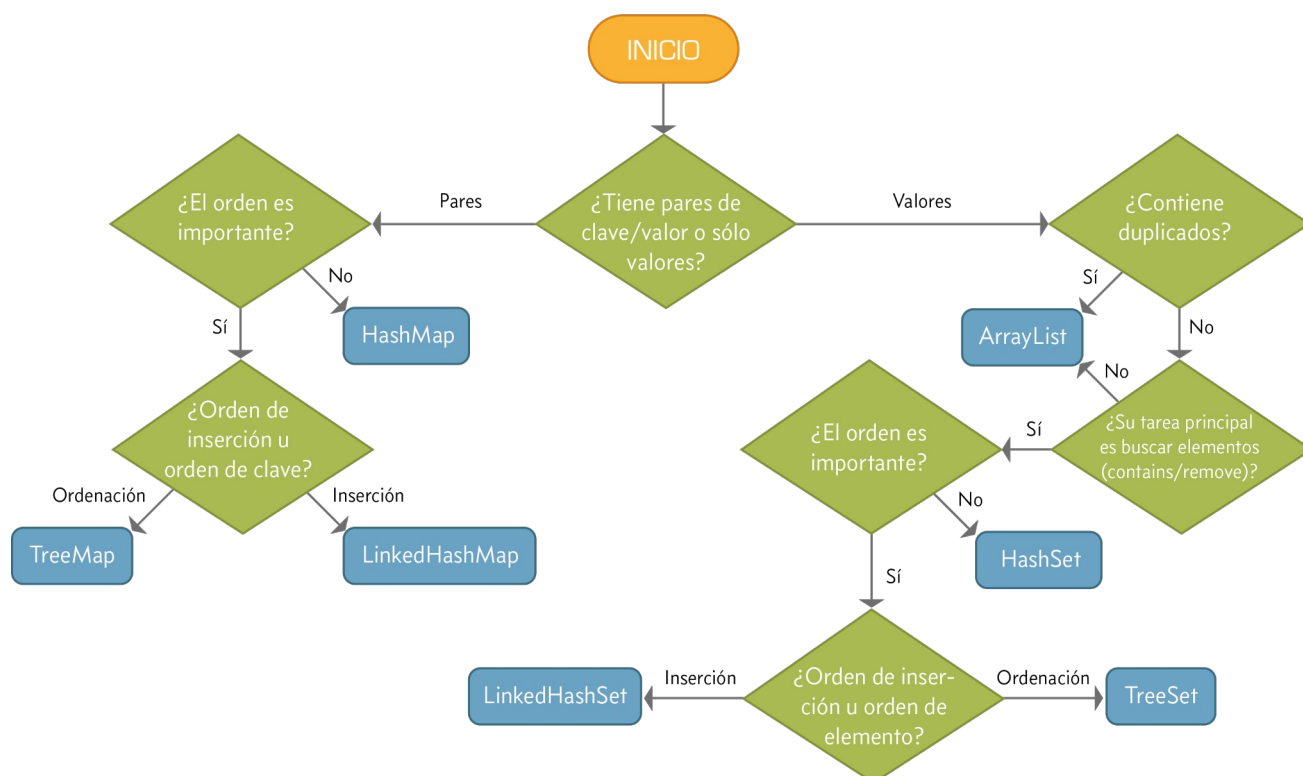
La elección de la estructura, o estructuras, de datos a emplear en la aplicación es absolutamente fundamental y tiene un gran impacto en el rendimiento de la misma, incluso en la funcionalidad. En la fase de diseño técnico debemos tomar decisiones acerca de qué estructuras emplear y porqué, con el objetivo de lograr la eficiencia y eficacia de la aplicación.

Algunos aspectos a tener presentes son: tiempo de ejecución requerido por la estructura, espacio de almacenamiento usado (y su tipo: estático o dinámico), cantidad de operaciones de escritura a realizar, cantidad de operaciones de lectura a realizar, tipología de los datos a almacenar y características o particularidades de los mismos así como la relación entre ellos.

¿Qué estructura de datos elegir?

El siguiente diagrama de flujo ofrece una forma de seleccionar el tipo de colección a emplear, en Java, según unos factores o criterios:

Diagrama de decisión para uso de colecciones Java



9. ALGORITMOS DE ORDENACIÓN

Los algoritmos de ordenación son una tipología de procedimientos fundamentales en la informática, los cuales consisten en organizar conjuntos de datos en función de un orden o criterio específico (creciente o decreciente, alfabéticamente, etcétera). Los algoritmos de ordenación permutan los elementos del conjunto de datos hasta conseguir dicho orden. Para ello se basan en dos operaciones básicas: la comparación y el intercambio. Existen muchos algoritmos de ordenación con diferentes ventajas e inconvenientes; en este apartado vamos a ver los más comunes.

9.1 Ordenación por inserción directa

Este algoritmo de ordenación consiste en lo siguiente: inicialmente, se ordenan los dos primeros elementos de la estructura de datos, luego se inserta el tercer elemento en la posición correcta respecto a los dos primeros. A continuación, se inserta el cuarto elemento en la posición correcta respecto a los tres primeros elementos ya ordenados, y así sucesivamente hasta llegar al último elemento. Veamos una implementación en Java:

```
public static void ordInsercionDirecta(int[] vector) {
    int i, k = 0;
    double x;

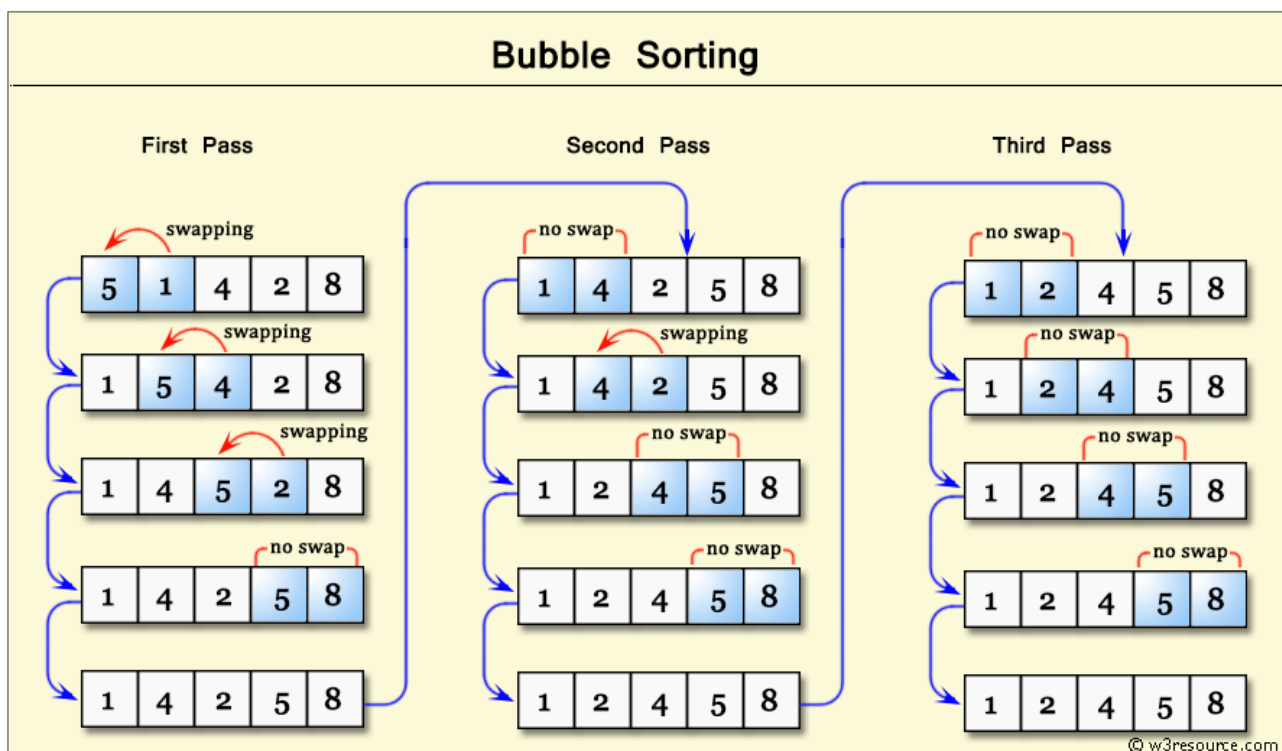
    // Empezamos desde el segundo elemento
    for (i = 1; i < vector.length; i++) {
        x = vector[i];
        k = i - 1;
        // Para k = - 1, hemos alcanzado el extremo izquierdo

        while (k >= 0 && x < matriz[k]) {
            // crea espacio para insertar
            vector[k + 1] = vector[k];
            k--;
        }
        vector[k + 1] = x; // insertamos x en su lugar
    }
}
```

9.2 Ordenación por el método de la burbuja

El algoritmo de la burbuja es uno de los métodos de ordenación más conocidos y uno de los primeros que aprenden los programadores. Consiste en comparar pares de elementos adyacentes en un array y, si están desordenados, intercambiarlos hasta que estén todos ordenados.

```
public static void burbuja(int[] A) {
    int i, j, aux;
    for (i = 0; i < A.length - 1; i++) {
        for (j = 0; j < A.length - i - 1; j++) {
            if (A[j + 1] < A[j]) {
                aux = A[j + 1];
                A[j + 1] = A[j];
                A[j] = aux;
            }
        }
    }
}
```



El tiempo de ejecución del algoritmo de la burbuja es del orden $O(n^2)$. Es uno de los peores algoritmos de ordenación en cuanto a tiempo de ejecución, solamente es

recomendable su uso para ordenar listas con un número pequeño de elementos.

9.3 Ordenación por QuickSort

Es el algoritmo de ordenación más rápido. Se basa en la técnica **divide y vencerás**, que consiste en ir subdividiendo el array en arrays más pequeños, y ordenar éstos. Para hacer esta división, se toma un valor del array como pivote, y se mueven todos los elementos menores que este pivote a su izquierda, y los mayores a su derecha. A continuación se aplica el mismo método a cada una de las dos partes en las que queda dividido el array. Su implementación es recursiva.

```
public static void quicksort(int A[], int izq, int der) {

    int pivote=A[izq]; // tomamos primer elemento como pivote
    int i=izq;         // i realiza la búsqueda de izquierda a derecha
    int j=der;         // j realiza la búsqueda de derecha a izquierda
    int aux;

    while(i < j){
        while(A[i] <= pivote && i < j) i++; // busca elemento mayor que pivote
        while(A[j] > pivote) j--;          // busca elemento menor que pivote
        if (i < j) {                       // si no se han cruzado
            aux= A[i];                     // los intercambia
            A[i]=A[j];
            A[j]=aux;
        }
    }

    A[izq]=A[j]; // se coloca el pivote en su lugar de forma que tendremos
    A[j]=pivote; // los menores a su izquierda y los mayores a su derecha

    if(izq < j-1)
        quicksort(A, izq, j-1); // ordenamos subarray izquierdo
    if(j+1 < der)
        quicksort(A, j+1, der); // ordenamos subarray derecho
}
```

10	40	7	9	15	27
----	----	---	---	----	----

Array original a ordenar

10	40	7	9	15	27
-----------	----	---	---	----	----

Se toma como pivote el primer elemento

10	40	7	9	15	27
-----------	-----------	---	----------	----	----

La búsqueda de izquierda a derecha encuentra el 40, mayor que pivote

i

j

y la búsqueda de derecha a izquierda encuentra el 9, menor que pivote

10	9	7	40	15	27
-----------	----------	---	-----------	----	----

Se intercambian

10	9	7	40	15	27
-----------	---	---	-----------	----	----

Continúa la búsqueda, se encuentra el valor 40 mayor que pivote y el

j

i

valor 7 menor que pivote, pero ya se han cruzado. Paramos.

7	9	10	40	15	27
---	---	-----------	----	----	----

subarray 1

subarray 2

Finalmente colocamos el pivote en su lugar. (Posición j), quedando el

array dividido en dos subarrays a los que les aplicará el mismo proceso

9.4 Ordenación por MergeSort

El método MergeSort es un algoritmo de ordenación recursivo con un número de comparaciones entre elementos del array mínimo. Su funcionamiento es similar al Quicksort, y está basado en la técnica divide y vencerás.

De forma resumida el funcionamiento del método MergeSort es el siguiente:

- Si la longitud del array es menor o igual a 1 entonces ya está ordenado.
- El array a ordenar se divide en dos mitades de tamaño similar.
- Cada mitad se ordena de forma recursiva aplicando el método MergeSort.
- A continuación las dos mitades ya ordenadas se mezclan formando una secuencia ordenada.

La ordenación MergeSort se puede implementar en Java de la siguiente forma:

```
public static void mergesort(int A[], int izq, int der){
    if (izq < der){
        int m=(izq+der)/2;
        mergesort(A, izq, m);
        mergesort(A, m+1, der);
        merge(A, izq, m, der);
    }
}
```

La operación principal de mezcla la realiza el método merge. Este método se puede implementar de varias formas, la más usual es la siguiente:

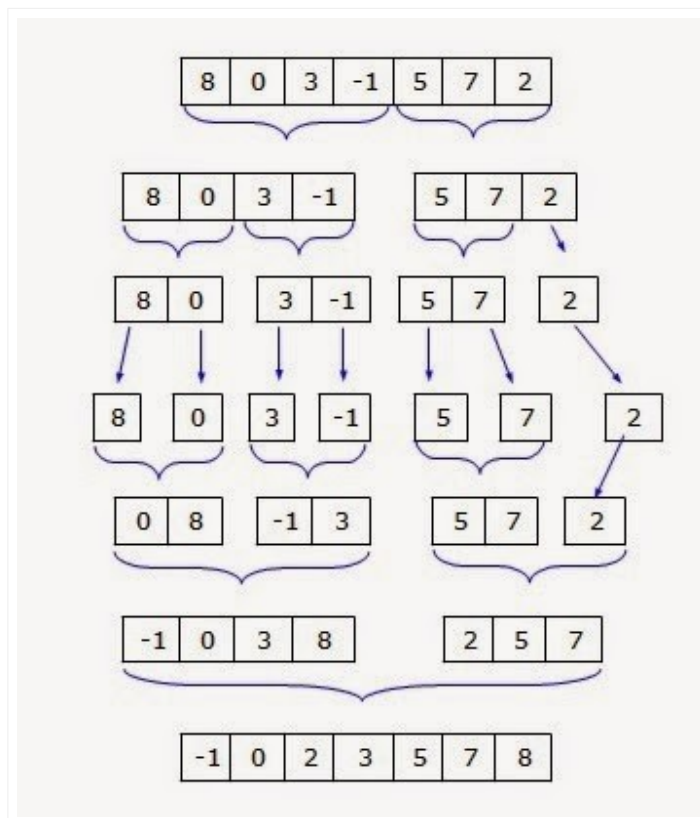
```
public static void merge(int A[], int izq, int m, int der){
    int i, j, k;
    int [] B = new int[A.length]; //array auxiliar
    for (i=izq; i<=der; i++) //copia ambas mitades en el array auxiliar
        B[i]=A[i];

    i=izq; j=m+1; k=izq;

    while (i<=m && j<=der) //copia el siguiente elemento más grande
        if (B[i]<=B[j])
            A[k++]=B[i++];
        else
            A[k++]=B[j++];

    while (i<=m) //copia los elementos que quedan de la
        A[k++]=B[i++]; //primera mitad (si los hay)
}
```

De forma gráfica podemos representar el funcionamiento del algoritmo de la siguiente forma:



El tiempo de ejecución promedio del método MergeSort es $O(n \log n)$.

9.5 Ordenación estándar en Java: el método .sort()

El método `.sort()`, internamente, implementa y usa 2 algoritmos famosos: quicksort y mergesort. Quicksort se usa cuando la estructura está formada por tipos primitivos, mientras que mergesort se emplea con estructuras que tienen objetos.

10. ALGORITMOS DE BÚSQUEDA

Los algoritmos de búsqueda, fundamentales en la algorítmica, son procedimientos diseñados para localizar un elemento, que cumpla un patrón o condición, dentro de una estructura de datos concreta. Por ejemplo, encontrar una persona concreta en una base de datos a partir de su DNI. En este apartado estudiaremos los 2 métodos de búsqueda tradicionales: la secuencial y la binaria (o dicotómica).

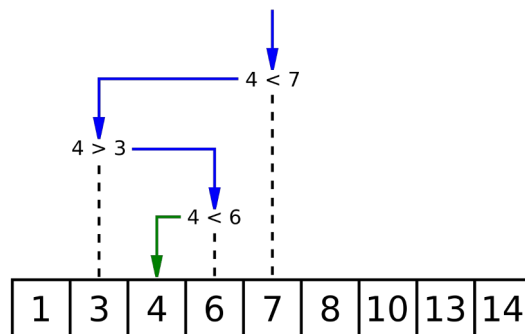
10.1 Búsqueda secuencial

Consiste en ir comparando el elemento a buscar con cada elemento del vector hasta encontrarlo o hasta que se llegue al final, esto hace que la búsqueda sea secuencialmente (de

ahí su nombre). La existencia se puede asegurar cuando el elemento es localizado, pero no podemos asegurar la no existencia hasta no haber analizado todos los elementos del vector. Se utiliza sin importar si el vector está previamente ordenado o no. Dada la sencillez del algoritmo, no pondremos aquí su implementación.

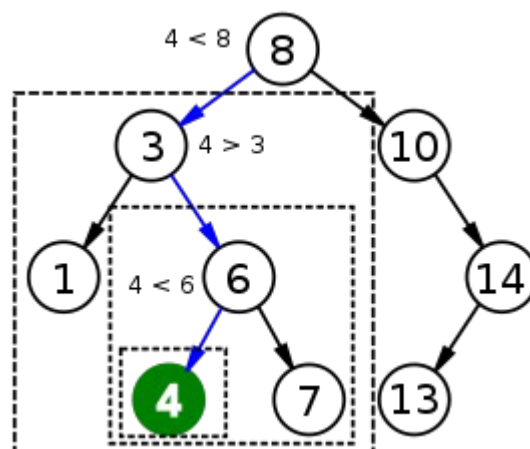
10.2 Búsqueda binaria (dicotómica)

Se utiliza cuando el vector en el que queremos determinar la existencia de un elemento está previamente ordenado. Este algoritmo reduce el tiempo de búsqueda considerablemente, ya que disminuye exponencialmente el número de iteraciones necesarias. En el peor de los casos el número máximo de comparaciones es $\lfloor \log_2(n) + 1 \rfloor$, donde n es el número de los elementos en el vector. Por ejemplo, en uno conteniendo 50.000.000 elementos, el algoritmo realiza como máximo 26 comparaciones.



```
import java.util.Arrays;
public class EjemploBinarySearch {

    public static void main(String[] args) {
        int[] array = { 10, 15, 20, 50, 100, 120, 200, 500, 600, 2222 };
        int result = Arrays.binarySearch(array, 400);
        System.out.println(String.format("Resultado %d", result));
    }
}
```



Es importante mencionar que el método **binarySearch** de la clase **Arrays** también requiere que el array que se pase como parámetro se encuentre **ordenado**, en caso contrario los resultados pueden ser erróneos. Por tanto, antes de llamar a **binarySearch** debemos ordenar la estructura de datos que se nos proporcione empleando algún método de ordenación.

11. OTROS ALGORITMOS

Además de los algoritmos de ordenación y búsqueda, fundamentales en la Informática, existen infinitos algoritmos de otras categorías, algunos de los cuales conviene conocer. En este apartado vamos a estudiar algunos de ellos.

11.1 Técnicas y estrategias para diseño de algoritmos

- Divide y vencerás (recursividad).
- Programación dinámica
- Algoritmos voraces
- Backtracking
- Ramificación y poda

Programación dinámica

Frecuentemente para resolver un problema complejo se tiende a dividir este en subproblemas más pequeños, resolver estos últimos (recurriendo posiblemente a nuevas subdivisiones) y combinar las soluciones obtenidas para calcular la solución del problema inicial; es decir, aplicar la técnica de “Divide y venderás” a través de la recursividad. Puede ocurrir que la división natural del problema conduzca a un gran número de subejemplares idénticos.

Si se resuelve cada uno de ellos sin tener en cuenta las posibles repeticiones, resulta un algoritmo ineficiente, cosa que sucede con la recursividad en el momento en que el tamaño del problema es grande; en cambio si se resuelve cada ejemplar distinto una sola vez y se conserva el resultado, el algoritmo obtenido es mucho mejor.

Esta es la idea de la programación dinámica: no calcular dos veces lo mismo y utilizar normalmente una tabla de resultados que se va rellenando a medida que se resuelven los subejemplares. La programación dinámica es un método ascendente. Se resuelven primero los subejemplares más pequeños y por tanto más simples. Combinando las soluciones se obtienen las soluciones de ejemplares sucesivamente más grandes hasta llegar al ejemplar original. Sin embargo, no todo son ventajas ya que resolver problemas con programación dinámica es complicado y requiere de un análisis muy profundo del problema.

La programación dinámica y los algoritmos voraces pueden aplicarse al mismo problema,

siendo especialmente indicadas para problemas de optimización; pero se diferencian en un factor: el algoritmo voraz es, generalmente, más eficiente pero no garantiza la optimalidad.

Algoritmos voraces

Son algoritmos que toman decisiones de corto alcance, basadas en información inmediatamente disponible, sin importar consecuencias futuras. Suelen ser bastante simples y se emplean sobre todo para resolver problemas de optimización, como por ejemplo, encontrar la secuencia óptima para procesar un conjunto de tareas por un computador, hallar el camino mínimo de un grafo, etc.

El nombre voraz proviene de que, en cada paso, el algoritmo escoge el mejor "pedazo" que es capaz de "comer" sin preocuparse del futuro. Nunca deshace una decisión ya tomada: una vez incorporado un elemento a la solución permanece ahí hasta el final; y cada vez que un elemento es rechazado, lo es para siempre.

Backtracking

Técnica ampliamente utilizada para resolver problemas a través de una búsqueda exhaustiva, generalmente sobre estructuras de datos de tipo árbol. La idea es encontrar la mejor combinación posible en un momento determinado, por eso, se dice que este tipo de algoritmo es una búsqueda en profundidad. Durante la búsqueda, si se encuentra una alternativa incorrecta, se retrocede hasta el paso anterior y se toma la siguiente alternativa.

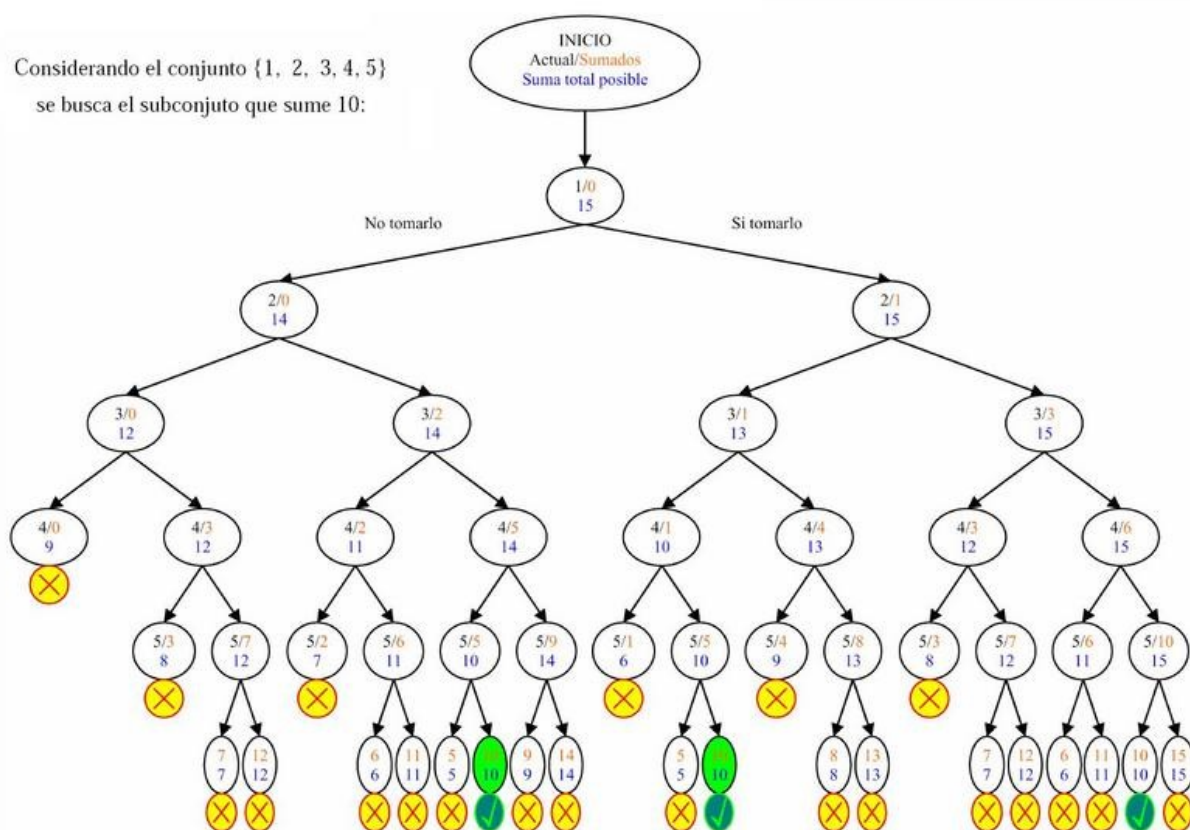
Cuando se han terminado las posibilidades, se vuelve a la elección anterior y se toma la siguiente opción (hijo [si nos referimos a un árbol]). Si no hay más alternativas la búsqueda falla. Ejemplos de aplicación: sudoku, movimientos de ajedrez, inteligencia artificial.

Ramificación y poda

Es una variante del Backtracking mejorado sustancialmente. El término (del inglés, Branch and Bound) se aplica mayoritariamente para resolver cuestiones o problemas de optimización.

La técnica de Ramificación y poda se suele interpretar como un árbol de soluciones, donde cada rama nos lleva a una posible solución posterior a la actual. La característica de esta técnica con respecto a otras anteriores (y a la que debe su nombre) es que el algoritmo se encarga de detectar en qué ramificación las soluciones dadas ya no están siendo óptimas, para «podar» esa rama del árbol y no continuar malgastando recursos y procesos en casos que se alejan de la solución óptima. Ejemplo gráfico:

Considerando el conjunto {1, 2, 3, 4, 5}
se busca el subconjunto que sume 10:



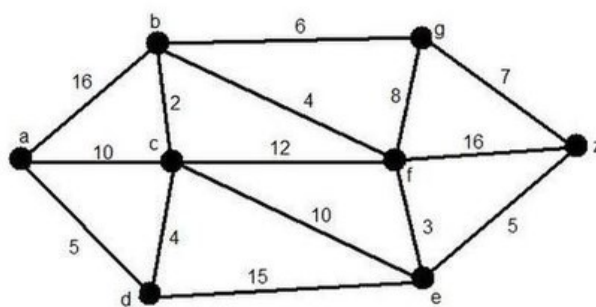
11.2 Algoritmos del camino más corto y recubrimiento mínimo

Son algoritmos muy utilizados en campos diversos, como por ejemplo: el enrutamiento en redes de comunicaciones, el cálculo de la ruta en un GPS, obtener la manera más económica de conectar por carretera varias ciudades de un recorrido, y un largo etcétera.

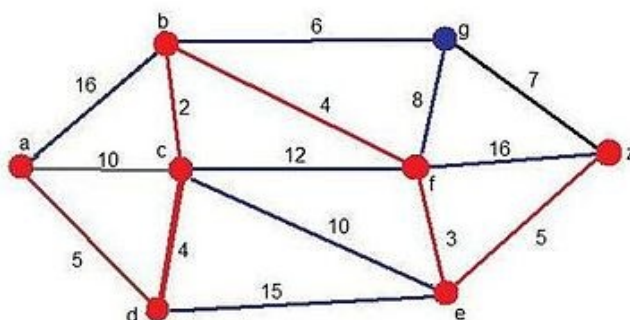
Los más importantes son: Dijkstra, Kruskal, Bellman-Ford y Ford-Fulkerson.

11.2.1 Algoritmo de Dijkstra

Es un algoritmo para la determinación del camino más corto, dado un vértice origen, hacia el resto de los vértices en un grafo que tiene pesos en cada arista. Usado, por ejemplo, en el cálculo de una ruta por GPS. Veamos la ruta más corta desde "a" hasta "z" en el siguiente grafo:



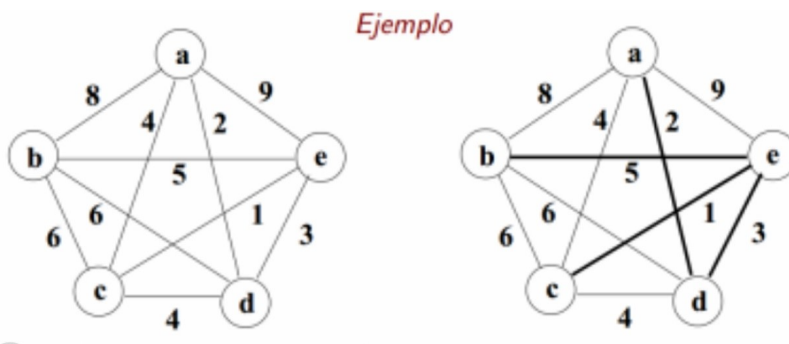
Tras varios pasos, concretamente en este ejemplo harían falta 7, llegamos a la solución:



Su implementación es muy compleja y no la vamos a ver aquí.

11.2.2 Algoritmo de Kruskal

Partiendo de un grafo conexo, no dirigido y ponderado, encuentra un árbol de expansión mínima. Es decir, es capaz de encontrar un subconjunto de las aristas que formen un árbol que incluya todos los vértices del grafo, donde el peso total de las aristas es el mínimo posible. Una de las aplicaciones más destacadas del árbol mínimo recubridor se encuentra en el ámbito de las telecomunicaciones, por ejemplo, en las redes de comunicación eléctrica, telefónica, etc. Los nodos representarían puntos de consumo eléctrico, teléfonos, aeropuertos o computadoras. Las aristas podrían ser cables de alta tensión, fibra óptica, rutas aéreas,...



12. COMPLEJIDAD COMPUTACIONAL

La Teoría de la Complejidad Computacional es la parte de la teoría de la computación que estudia los recursos requeridos durante el cálculo para resolver un problema. Así, un cálculo difícil requerirá más recursos que uno de menor dificultad. Los recursos comúnmente estudiados son el tiempo (número de pasos de ejecución de un algoritmo para resolver un problema) y el espacio (cantidad de memoria utilizada para resolver un problema).

Un algoritmo que resuelve un problema pero que tarda mucho en hacerlo difícilmente será de utilidad. Igualmente un algoritmo que necesite varios gigabytes de memoria probablemente no será utilizado.

Orden de complejidad

Para analizar el rendimiento de un algoritmo de un modo sencillo, se usa la notación Big O, lo cual es una notación matemática para expresar cuanto tarda un algoritmo en ejecutarse atendiendo sólo a grandes rasgos su eficiencia, generalmente centrándonos en el caso promedio o peor. En concreto se usa para expresar de forma abstracta la velocidad de procesamiento de un algoritmo, atendiendo a cómo aumentaría su tiempo de ejecución en función de un aumento significativo del tamaño de los datos de entrada.

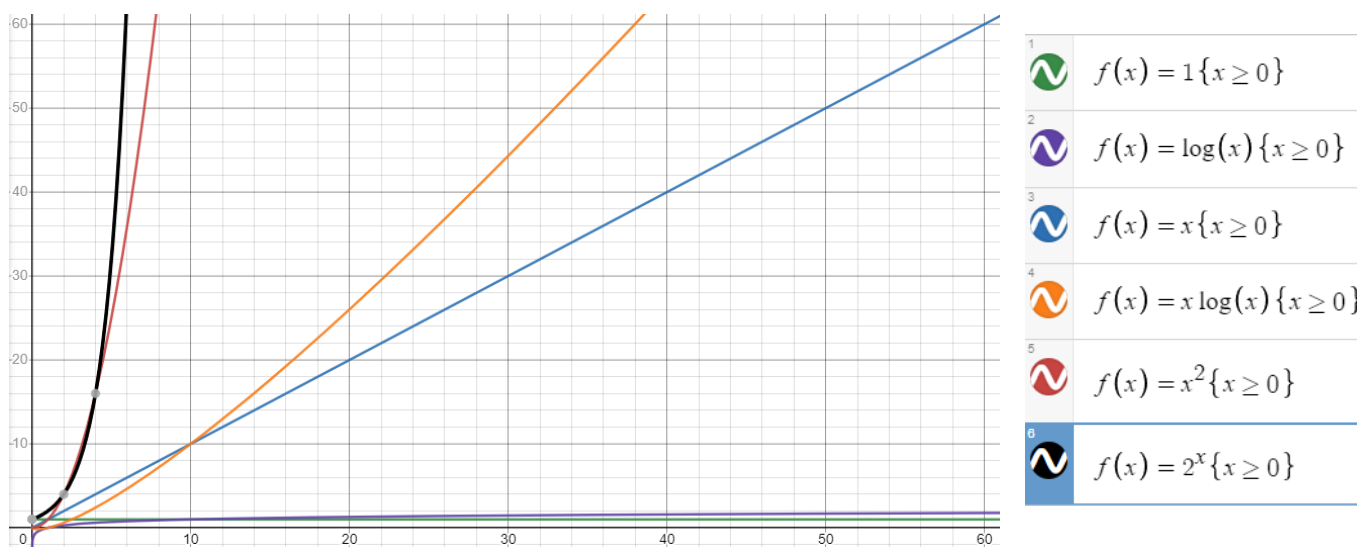
La O viene de clasificar el ORDEN de magnitud de la función a analizar, resumiéndose en que nos sirve para determinar cómo un algoritmo escala en complejidad tanto en tiempo como en espacio necesario, en función de su tamaño de entrada. Podemos resumir la nota dada a unos pocos valores, ya que descartamos las constantes y sólo atendemos a unos pocos casos según el orden de magnitud de crecimiento, y sin importar por ejemplo el detalle de la inclinación de cada curva y sólo la forma de comportamiento de crecimiento. En resumen, nos podemos quedar con que tenemos:

- $O(1)$ - tiempo **constante**: es el mejor resultado, y quiere decir que el tiempo de ejecución no varía conforme aumenta el tamaño de los datos de entrada, y la respuesta siempre tarda lo mismo sin importar la magnitud de entrada.
- $O(\log n)$ - tiempo **logarítmico**: una forma de crecimiento que crece al inicio pero tiende a estabilizarse conforme aumentan el tamaño de entrada, por lo que es una buena nota para un algoritmo ya que no tiende a resentirse.
- $O(n)$ - tiempo **lineal**: el crecimiento es lineal en tanto el tiempo de ejecución es cada vez mayor de modo proporcional a cómo se incrementa el tamaño de la entrada. Por lo que si tenemos el doble de elementos de entrada, tardará el doble, aunque despreciamos realmente la pendiente de la misma y sólo nos quedamos con que aumenta de forma lineal.
- $O(n \cdot \log n)$ - tiempo **lineal-logarítmico** (mal llamado “casi lineal”): es un buen coste, en

términos generales. El algoritmo de Quicksort se engloba en este coste computacional.

- $O(n^2)$ - tiempo **cuadrático** (o polinómico): el crecimiento es de forma potencial por lo que será un algoritmo a evitar ya que para valores pequeños de entrada el tiempo será asumible, pero conforme aumente el tamaño de los datos de entrada el tiempo tenderá a ser muy elevado y es probable que el procesador se quede inoperativo.
- $O(a^n)$ - tiempo **exponencial**, por ejemplo $O(2^n)$: el crecimiento es de forma exponencial, por lo que el coste computacional y espacial se dispara con tamaños de entrada pequeños. Debe evitarse.
- $O(n!)$ - tiempo **factorial**: el crecimiento es factorial, por lo que rápidamente tiende a valores imposibles de tratar, en lo que sería una recta vertical. Evitarse a toda costa.

Para entender mejor esto, adjuntamos una gráfica comparativa:



A nivel práctico, por ejemplo, si consultamos la documentación de MSDN para la función RemoveAt de una SortedList nos dice qué es una operación $O(n)$:

Remarks

The index sequence is based on the sort sequence. When an element is added, the index sequence adjusts accordingly. When an element is removed, the indexing also adjusts a change as elements are added or removed from the [SortedList](#) object.

In collections of contiguous elements, such as lists, the elements that follow the collection is indexed, the indexes of the elements that are moved are also updated. Conceptually grouped into buckets, such as a hash table.

This method is an $O(n)$ operation, where n is [Count](#).

Y si consultamos el API de Java, en la clase Arrays, el método “sort” nos indica en su descripción:

*Implementation note: The sorting algorithm is a Dual-Pivot Quicksort by Vladimir Yaroslavskiy, Jon Bentley, and Joshua Bloch. **This algorithm offers $O(n \log(n))$ performance on many data sets that cause other quicksorts to degrade to quadratic performance, and is typically faster than traditional (one-pivot) Quicksort implementations.***

¿Qué nos indica esa descripción acerca de la complejidad del método “sort”?...

Veamos algunos **ejemplos** en Java:

Coste lineal $O(n)$: código para búsqueda secuencial (recorrer un vector y comprobar cada posición).

Coste logarítmico $O(\log n)$: ver ejemplo de “búsqueda binaria”.

Coste cuadrático $O(n^2)$:

```
void buscar(int[] vector){
    for(int i=0;i<vector.size;i++)
        for(int j=0;j<vector.size;j++)
            if(i!=j && vector[i] == vector[j])
                System.out.println("Elemento " + vector[i] + "repetido!");
}
```

Coste exponencial $O(2^n)$: calcular el número de fibonacci.

```
int fibonacci(int numero){
    if(numero <= 1)
        return numero;
    else
        return fibonacci(numero - 2) + fibonacci(numero - 1);
}
```